异步任务管线与取消语义

摘要：文档处理在入库时耗时较长，若同步执行会让接口长时间无响应。本文讨论把处理改造成异步管线之后引入的一整套新问题：任务状态如何表达、取消如何生效、失败如何归因、以及进程重启之后中间态任务怎么办。

一章　受理与执行分离。受理与执行必须分离否则接口会被长任务拖住。受理只做校验与入队，真正的解析、分块与编码交给后台执行，接口立刻返回任务标识。用户随后通过状态查询获知进度，整条链路对交互延迟的敏感度大幅下降。

二章　状态机。状态至少要有：已上传、解析中、分块中、编码中、就绪、失败、已取消。把“处理中”作为一个状态会导致前端无法给出有意义的进度，也让排障时无法判断卡在哪个阶段。

三章　失败归因。状态机必须显式记录失败原因而不是只记录失败。仅有失败标记时，用户看到的只是一句“处理失败”，而实际原因可能是文件损坏、编码服务不可用或磁盘空间不足。这些原因的修复动作完全不同，因此失败原因必须落到可读字段并展示给用户。

四章　取消的必要性。长任务的取消不是可选项：用户可能上传了错误的文件，或发现解析结果不合预期。若不能取消，用户只能等待它跑完，在此期间设备资源被无效任务占用，后续操作一并变慢。

五章　取消语义。取消必须被记录为已取消而不是失败。这两者的区别不只是措辞：失败会触发重试与告警，而取消是用户的正常意图，不应产生任何错误提示，也不应计入失败率。把取消记为失败会污染监控并让用户以为自己操作有误。

六章　协作式取消的实现。协作式取消要求任务在安全点主动检查停止标志。在 Ｐｙｔｈｏｎ 这类运行时里，落盘、编码、写数据库都可能是不可中断的原子步骤，因此停止标志应当在每两个步骤之间检查，并在检查到之后主动清理已产生的中间产物。

七章　中间产物的清理。取消之后若不清理，磁盘上会留下半个索引与若干临时文件。这些残留不仅占空间，还会在下次处理同一文档时造成标识冲突。清理必须在同一条取消路径里完成，不能依赖后续的定期清扫。

八章　进度上报。进度回调不能改变任务状态否则会掩盖真正的终态。进度是观测数据，状态是权威数据，两者混用会出现“进度显示已完成但状态仍是处理中”这类自相矛盾的现象，让前端无从判断该以谁为准。

九章　重启与恢复。重启后必须能识别并清理处于中间态的任务。进程崩溃或重启时，正在处理的任务会永久停在中间态。启动时应当扫描这些任务并标记为中断，允许用户重新触发，而不是让它们永远显示成“处理中”。

十章　并发度。后台任务的并发度必须设上限，否则一次批量上传会把编码服务压满，导致交互请求也被拖慢。合理做法是给批量任务单独排队并限制并发，把交互路径的容量预留出来。

十一章　与检索的关系。文档在编码完成之前不应进入可检索集合，否则会出现“刚上传就能检索到、但内容不完整”的中间状态。就绪标记应当作为检索过滤器的一部分，而不是仅用于前端展示。

十二章　幂等性。用户可能对同一文件重复上传，或在上传失败后重试。处理任务需要按内容指纹判断是否已有等价文档，避免重复编码与重复入库，同时要保证同一批重复请求不会并发跑两遍。

十三章　观测。需要记录的指标包括各阶段耗时、失败率与取消率。其中取消率是判断交互设计是否合理的重要信号：取消率异常升高通常说明用户被引导上传了不需要的文件，而不是任务本身有问题。

十四章　与取消相关的资源回收。取消发生在编码阶段时，已经写入的向量需要一并删除，否则数据库里会出现指向不存在文件的记录。清理动作应当与取消标记在同一个事务里完成，中途崩溃时才能通过重放保持一致，而不是留下半清理状态。

十五章　取消与队列。任务若仍在队列里等待，取消应当直接从队列移除；若已经在执行，则只能通过停止标志协作式退出。两种情况的处理路径不同，把它们合并成一条路径会出现“取消了但仍占用

资源直到跑完”的现象。

十六章　上传与解析的边界。文件落盘成功但解析失败是最常见的失败形态，此时任务应当保留在失败态并允许重试解析，而不是要求用户重新上传。把这两种动作合并会让用户为一次可恢复的解析错误重复付出上传成本。

十七章　批量与单条的差异。单条上传时用户会盯着进度，因此进度必须细；批量上传时用户关心的是整体完成情况，逐条进度反而造成信息过载。因此状态接口应当同时提供单条明细与批量汇总两个视图。

十九章　回滚的完整性。取消之后必须回滚已写入的中间产物。取消请求可能在任意阶段到达，因此每一阶段都要能回答「我到目前为止写了什么」。把回滚逻辑写在同一个阶段内部，比在最外层写一个大回滚更可靠，因为外层无法知道内层到底提交到了哪一步。

二十章　状态查询的成本。状态查询接口本身也必须足够快否则轮询会放大负载。前端通常会以固定间隔轮询，若状态查询要走数据库并把任务的全部产物元数据都读出来，那么十几个任务同时上传就会让查询本身成为瓶颈。因此状态查询应当只读一个轻量的内存状态，把重查询留给详情页。

二十一章　轮询与推送。轮询实现简单但有固定的延迟与额外负载；推送及时但需要处理断线与重连。在多任务场景下，常见折中是只对「仍在处理中」的任务保持轮询，一旦进入终态就停止轮询并释放连接。

二十二章　队列公平性。任务队列的公平性决定了批量任务会不会饿死交互任务。若队列严格先进先出，一次批量上传会把后续所有交互任务排在后面，用户感知为“整个系统变慢了”。可行做法是给交互路径独立的队列而不是同队列里加优先级，因为同队列的优先级在面对大量批任务时仍会退化成排队。

二十三章　并发上限的取值。并发上限应当按最慢阶段的容量来定，而不是按　ＣＰＵ　核数。若最慢阶段是需要　ＧＰＵ　的编码，那么并发数超过显存能容纳的批次数只会导致排队与显存抖动。

二十四章　中间产物的位置。中间产物应当放在可被整体清理的目录下，并在文件名里带上任务标识。若把临时文件与最终产物混放，取消之后就无法安全地批量删除，只能逐个判断。

二十五章　失败重试的边界。解析失败可以重试，因为输入没有变；编码失败若源于资源不足，重试只会再次失败并加剧拥塞。因此重试策略必须区分「可重试」与「不可重试」两类原因，并把重试次数与退避间隔写进任务记录。

二十六章　用户可见的进度语义。进度百分比应当是「已完成阶段数」而不是「已用时间」，因为后者在慢速阶段会停在某个数字上不动，看起来像卡死。若无法给出阶段级进度，宁可显示不确定动画也不要显示假进度。

二十七章　通知与幂等。任务完成后的通知可能重复送达，因此下游处理必须幂等：同一任务标识的完成事件只能生效一次。这在重启与重试同时存在时尤其容易出问题。

二十八章　资源与配额。单用户场景下也需要配额，否则一次误操作把所有文件都上传一遍就会长时间占满设备。配额应当按未完成任务数而不是按历史总数来限制。

二十九章　观测。除各阶段耗时与失败率外，还应当记录队列等待时长。等待时长与处理时长是两种不同的慢：前者说明容量不足，后者说明算法太慢，而用户看到的现象完全相同，只有指标能区分。

三十章　与检索的耦合。任务完成事件应当触发索引刷新，但刷新频率必须受限，否则连续上传十篇文档会触发十次重建。常见折中是合并短时间内的多次完成事件，只做一次刷新。

三十二章　任务的可见范围。中间态任务应当对用户可见，包括它正在使用多少资源、预计还需要多久、以及是否可以安全取消。把中间态做成黑盒，用户只能在两个极端之间选择：等待或强杀进程。

三十三章　与文档生命周期的衔接。文档从上传到可检索之间跨越多个阶段，而这些阶段的失败都应当汇总到同一个状态上。若每个阶段各自维护状态，用户会看到互相矛盾的信息，例如列表页显示就绪而详情页显示失败。

三十四章　超时与判定。长任务必须有整体超时，且超时后要落到明确的终态，而不是停在处理中。超时阈值应当按最慢阶段的实测分位数来定，设得过紧会把正常的大文档误判为失败。

三十五章　日志与可追溯。任务日志应当带上任务标识与阶段名，并且保留足够长时间以支持事后排查。在单机部署里日志往往同时是唯一的事后证据，因此不能只写“出错了”。

三十六章　结论。把长任务改成异步并不是把同步调用包一层就完了，它要求状态、取消、失败、重启四条路径都被显式设计。这四条路径缺任何一条，用户都会在某个时刻遇到无法解释的等待或无法恢复的中间态。